



Orviohub Pricing Implementation Plan

Pricing strategy

Orviohub should use a **workspace-based subscription model**, not a purely user-based model.

The customer pays for a workspace and receives access to one or more products according to the plan.

The core rule is:

Free users can have only one workspace and one application. Standard and Premium users can use more workspaces and applications according to their plan.

This structure fits your long-term goal of creating a Zoho-like platform while keeping the free plan useful but limited.

Local Nigerian POS and inventory products commonly position entry-level plans around ₦3,000–₦12,000 monthly, while broader cloud POS products may reach ₦15,000–₦50,000 monthly depending on branches, users, and advanced features. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)

1. Recommended plans

Plan overview

Feature	Free	Standard	Premium
Price per month	₦0	₦7,500	₦20,000
Annual price	₦0	₦75,000	₦200,000
Workspaces	1	3	10
Applications	1	3	All available
Members per workspace	2	10	50
Branches per workspace	1	3	10
Products	500	5,000	25,000
Monthly transactions	300	5,000	25,000
Basic reports	Yes	Yes	Yes
Advanced reports	No	Yes	Yes

Feature	Free	Standard	Premium
CSV import/export	Limited	Yes	Yes
Customer credit	Basic	Yes	Yes
Supplier management	Basic	Yes	Yes
Barcode support	Basic	Yes	Yes
Offline sales	Limited	Yes	Yes
Stock counts	Basic	Yes	Advanced
Audit logs	7-day history	90-day history	1-year history
Support	Self-service	Email and WhatsApp	Priority support
API access	No	Limited	Yes
Custom roles	No	Limited	Yes
Multiple branches	No	Yes	Yes
Product activation	One app	Up to three apps	All available apps

The exact prices should be validated with pilot customers. The figures above are a practical starting point, not permanent pricing.

2. Free plan

Purpose

The Free plan should allow an individual or very small business to experience Orviohub without payment.

It should be useful enough to demonstrate value but limited enough to encourage serious businesses to upgrade.

Free-plan rules

A Free user can have:

- One Orviohub account.
- One workspace.
- One enabled application.
- One branch.
- Two members.
- 500 products.
- 300 transactions per month.
- Basic reports.

- Basic customer credit.
- Basic supplier records.
- Limited offline sales.
- Limited audit history.

Free plan limitations

The Free plan should not include:

- Multiple workspaces.
- Multiple applications.
- Multiple branches.
- Advanced reports.
- Custom roles.
- API access.
- Priority support.
- Advanced exports.
- Long audit retention.
- Large transaction history.
- Complex product variants.
- Advanced stock controls.

Free-plan experience

When a Free user tries to:

- Create another workspace.
- Activate another application.
- Add a second branch.
- Add a third member.
- Exceed the transaction limit.

Show a clear upgrade prompt.

Example:

Your Free plan includes one workspace and one application. Upgrade to Standard to manage additional workspaces and products.

Do not block access to existing data immediately. The user should be able to view data and upgrade.

3. Standard plan

Purpose

Standard should be the main plan for growing Nigerian small businesses.

It should support:

- More than one workspace.
- Multiple Orviahub applications.
- Multiple staff members.
- Multiple branches.
- Larger product catalogues.
- Better reports.
- Regular customer support.

Standard plan rules

A Standard customer can have:

- Up to three workspaces.
- Up to three applications across all workspaces.
- Ten members per workspace.
- Three branches per workspace.
- 5,000 products per workspace.
- 5,000 transactions per month per workspace.
- Full basic inventory functionality.
- Barcode support.
- Offline sales.
- Customer and supplier balances.
- CSV import and export.
- Advanced reports.
- Ninety days of audit history.
- Email and WhatsApp support.

Standard application allocation

There are two possible models.

Model A: Account-wide application limit

The customer can activate three total applications across all workspaces.

Example:

```
Workspace A:
- Inventory
- Task Management

Workspace B:
- Inventory

Workspace C:
- CRM
```

This uses four product activations and exceeds the Standard limit.

Model B: Workspace-level application limit

Each workspace can activate up to three applications.

This is more generous but can become expensive if customers create many workspaces.

Recommendation

Use:

```
Standard:
- Maximum 3 workspaces
- Maximum 3 active applications per workspace
```

Then separately control overall usage through:

- Member limits.
- Product limits.
- Transaction limits.
- Branch limits.
- Storage limits.

This is easier for customers to understand than a single global application counter.

4. Premium plan

Purpose

Premium should be designed for established businesses and customers using Orviohub as a broader operating platform.

It should support:

- More workspaces.
- Many applications.
- Larger teams.
- Multiple branches.
- Advanced operational reporting.
- Longer audit retention.
- Better support.
- API access.
- Custom roles.
- More integrations.

Premium plan rules

A Premium customer can have:

- Up to ten workspaces.
- All available applications.
- Fifty members per workspace.
- Ten branches per workspace.
- 25,000 products per workspace.
- 25,000 transactions per month per workspace.
- Advanced reports.
- Full audit history for one year.
- API access.
- Custom roles.
- Priority support.
- Advanced exports.
- Product-level configuration.
- Multi-workspace reporting in a later release.

Enterprise consideration

Do not build an Enterprise plan too early, but reserve the architecture for one.

Possible future Enterprise features:

- Unlimited workspaces.
- Unlimited members.
- Unlimited branches.
- Dedicated support.
- Service-level agreement.
- Custom data retention.
- SSO.
- SAML.
- SCIM.
- Dedicated environment.
- Custom integrations.
- Data residency requirements.
- Negotiated pricing.

5. Important pricing distinction

Do not confuse these concepts:

```
Workspace limit
Application limit
Member limit
Branch limit
Product limit
Transaction limit
Storage limit
API limit
```

Each should be enforced separately.

Example:

```
Plan: Standard
Workspaces: 3
Applications per workspace: 3
Members per workspace: 10
Branches per workspace: 3
Products per workspace: 5,000
Transactions per month: 5,000
```

This gives you flexible pricing without making the system difficult to explain.

6. Billing model

Billing owner

Billing should belong to the account or billing customer, while entitlements are applied to workspaces.

Recommended structure:

```
Billing account
  ↓
Subscription
  ↓
Plan
  ↓
Workspace entitlements
  ↓
Product access
```

Example

```
Billing account: Code X
Plan: Standard

Workspace A:
- Inventory: Active
- Task Management: Active

Workspace B:
- Inventory: Active

Workspace C:
- Not activated
```

Why not attach billing directly to individual users?

Because:

- Businesses pay for workspaces.
- Employees can join without paying individually.
- One owner may operate multiple businesses.
- Products belong to workspaces.
- Subscription limits are easier to manage at workspace level.

7. Pricing entities

plans

```
_id
key
name
description
monthlyPrice
annualPrice
currency
billingInterval
status
createdAt
updatedAt
```

Example:

```
{
  "key": "standard",
  "name": "Standard",
  "monthlyPrice": 750000,
  "annualPrice": 7500000,
  "currency": "NGN",
  "billingInterval": "monthly",
  "status": "active"
}
```

Store money in the smallest currency unit:

```
₦7,500 = 750000 kobo
```

planEntitlements

```
_id
planId
featureKey
limitValue
limitType
isEnabled
metadata
createdAt
updatedAt
```

Examples:

```
workspace.max_count = 3
workspace.max_apps_per_workspace = 3
workspace.max_members = 10
```

```
inventory.max_products = 5000
inventory.max_transactions_monthly = 5000
inventory.offline_sales = true
api.access = false
```

billingAccounts

```
_id
ownerUserId
billingEmail
provider
providerCustomerId
currency
status
createdAt
updatedAt
```

subscriptions

```
_id
billingAccountId
planId
provider
providerSubscriptionId
status
currentPeriodStart
currentPeriodEnd
cancelAtPeriodEnd
trialStart
trialEnd
gracePeriodEnd
createdAt
updatedAt
```

workspaceEntitlements

```
_id
workspaceId
planId
featureKey
limitValue
source
status
effectiveFrom
effectiveUntil
createdAt
updatedAt
```

The entitlement service should resolve permissions from the active plan rather than hardcoding plan checks throughout product code.

usageCounters

```
_id  
billingAccountId  
workspaceId  
productKey  
featureKey  
periodStart  
periodEnd  
usageValue  
limitValue  
updatedAt
```

Examples:

```
inventory.transactions  
inventory.products  
workspace.members  
workspace.branches  
storage.files  
api.requests
```

billingEvents

```
_id  
provider  
providerEventId  
eventType  
billingAccountId  
subscriptionId  
payload  
processedAt  
status  
errorMessage  
createdAt
```

Make provider event IDs unique to prevent duplicate processing.

8. Plan entitlement service

Do not write this everywhere:

```
if (plan === "premium") {  
  // allow feature  
}
```

Instead, create a reusable entitlement service.

Example API

```
await entitlementService.require({
  billingAccountId,
  workspaceId,
  productKey: "inventory",
  featureKey: "inventory.offline_sales",
});
```

Check limit

```
const result = await entitlementService.checkLimit({
  billingAccountId,
  workspaceId,
  productKey: "inventory",
  featureKey: "inventory.max_products",
  requestedAmount: 1,
});
```

Example response:

```
{
  "allowed": false,
  "featureKey": "inventory.max_products",
  "currentUsage": 5000,
  "limit": 5000,
  "remaining": 0,
  "upgradePlan": "premium"
}
```

Entitlement decision order

1. Is billing account active?
2. Is subscription active, trialing, or within grace period?
3. Is workspace active?
4. Is product enabled?
5. Is feature included in plan?
6. Is usage below limit?
7. Is user authorized?
8. Allow or reject action.

Authorization and entitlements are different:

Authorization:
Can this user perform the action?

Entitlement:
Does this paid plan include the action?

Both must pass.

9. Usage enforcement

When to check limits

Check limits when:

- Creating a workspace.
- Activating an application.
- Adding a member.
- Creating a branch.
- Creating a product.
- Importing products.
- Creating a sale.
- Uploading storage.
- Calling the API.
- Generating exports.

Hard and soft limits

Hard limit

Action is blocked after limit is reached.

Use for:

- Number of workspaces.
- Number of applications.
- Number of members.
- Number of branches.
- Number of products.
- Number of transactions.

Soft limit

Action is allowed, but user is warned.

Use for:

- Approaching storage limit.
- Approaching monthly transaction limit.

- Approaching API request limit.
- Approaching report history limit.

Usage warnings

Send warnings at:

- 50% usage.
- 80% usage.
- 90% usage.
- 100% usage.

Example:

You have used 4,500 of 5,000 monthly Inventory transactions. Upgrade to Premium for a higher limit.

10. Pricing user journeys

Journey A: Free user signs up

```
Landing page
↓
Create account
↓
Create workspace
↓
Choose Inventory
↓
Free plan automatically assigned
↓
Inventory onboarding
↓
Dashboard
```

Free account state:

```
Workspaces: 1 of 1
Applications: 1 of 1
Members: 1 of 2
Branches: 1 of 1
```

Journey B: Free user tries to create a second workspace

```
Workspace switcher
↓
Create workspace
↓
System checks entitlement
↓
Limit reached
↓
Upgrade modal
↓
Compare Standard and Premium
↓
Choose plan
↓
Pay
↓
Payment verified
↓
Standard entitlement activated
↓
Continue workspace creation
```

Journey C: Free user tries to activate Task Management

```
Application launcher
↓
Select Task Management
↓
System checks app limit
↓
Limit reached
↓
Upgrade prompt
↓
Select Standard
↓
Complete payment
↓
Task Management activated
↓
Task Management onboarding
```

Journey D: Standard user creates another workspace

```
Workspace selector
↓
Create workspace
↓
System checks 3-workspace limit
↓
```

```
If below limit:  
    create workspace  
If at limit:  
    display upgrade prompt
```

Journey E: Standard user activates fourth application

```
Application launcher  
↓  
Select fourth application  
↓  
System checks application limit  
↓  
Display plan limitation  
↓  
Upgrade to Premium  
↓  
Payment  
↓  
Entitlement updated  
↓  
Product activated
```

Journey F: Subscription payment fails

```
Provider sends failed-payment webhook  
↓  
Verify webhook  
↓  
Update subscription to past_due  
↓  
Send notification  
↓  
Start grace period  
↓  
Retry payment  
↓  
If successful: active  
If unsuccessful: restricted or suspended
```

Journey G: User downgrades

```
Billing settings  
↓  
Select downgrade  
↓  
System calculates conflicts  
↓  
Show affected workspaces/apps/features  
↓  
User removes or archives excess resources
```



```
↓  
Confirm downgrade  
↓  
Downgrade scheduled for period end  
↓  
New plan becomes active
```

Do not automatically delete data during downgrade.

Journey H: User cancels

```
Billing settings  
↓  
Cancel subscription  
↓  
Show end date  
↓  
Ask cancellation reason  
↓  
Confirm cancellation  
↓  
Subscription remains active until period end  
↓  
Workspace enters restricted state after period ends
```

11. Pricing screens

Public pricing page

URL:

```
orviohub.com/pricing
```

Display:

- Free.
- Standard.
- Premium.
- Monthly/annual toggle.
- Feature comparison.
- Application limits.
- Workspace limits.
- Member limits.
- Branch limits.
- Transaction limits.

- Support differences.
- FAQ.
- Start buttons.

In-app billing page

URL:

```
app.orviohub.com/billing
```

Display:

- Current plan.
- Subscription status.
- Billing account.
- Current period.
- Next charge.
- Usage.
- Payment method.
- Billing history.
- Upgrade.
- Downgrade.
- Cancel.
- Update billing information.

Upgrade modal

Show:

- Current plan.
- Selected plan.
- Added capabilities.
- New price.
- Billing interval.
- Effective date.
- Proration or immediate charge.
- Confirmation button.

Usage page

Display:

Workspaces	1 / 3
Applications	2 / 3
Members	7 / 10
Branches	2 / 3
Products	2,800 / 5,000
Transactions	3,900 / 5,000

Use progress indicators and plain explanations.

Limit-reached page

The page should include:

- What limit was reached.
- Current usage.
- Current plan.
- Plans that remove the limitation.
- Upgrade button.
- Continue without upgrade.
- Contact support.

12. Paystack implementation

Paystack supports recurring charges and subscriptions, including monthly and annual intervals. [\[5\]](#) [\[6\]](#)

Recommended flow

1. User selects plan.
2. Frontend requests payment initialization from Fastify.
3. Fastify creates or retrieves billing account.
4. Fastify initializes Paystack transaction.
5. Frontend redirects user to Paystack checkout.
6. Paystack returns reference.
7. Fastify verifies transaction server-side.
8. Paystack webhook is received.
9. Webhook signature is verified.
10. Billing event is stored.

11. Subscription status is updated.
12. Entitlements are activated.
13. User is redirected to success page.

Important payment rules

- Do not activate a paid plan based only on frontend redirect.
- Verify the transaction through Paystack.
- Verify and deduplicate webhooks.
- Store provider customer and subscription identifiers.
- Handle payment failures.
- Handle renewals.
- Handle cancellations.
- Handle chargebacks where applicable.
- Record all billing events.

Paystack plans

Create provider plans for:

```
orviahub_standard_monthly
orviahub_standard_annually
orviahub_premium_monthly
orviahub_premium_annually
```

Keep your internal plan IDs separate from provider plan IDs.

13. Subscription state machine

States

```
trialing
active
past_due
grace_period
cancelled
expired
suspended
```

Transitions

```
trialing → active
trialing → expired

active → past_due
past_due → active
past_due → grace_period
grace_period → active
grace_period → suspended

active → cancelled
cancelled → expired
```

Grace period recommendation

Use a short grace period after payment failure:

- Day 0: Payment fails.
- Day 0: Send notification.
- Day 2: Retry.
- Day 5: Send warning.
- Day 7: Restrict new resource creation.
- Day 14: Move workspace to read-only or limited access.
- Data remains preserved.

Exact timing should be validated against customer expectations and provider behavior.

14. What happens when a subscription expires?

Do not immediately delete data.

Recommended restrictions

Read-only access

Allow:

- View existing products.
- View reports.
- Export data.
- View customers.
- View transactions.
- Manage billing.
- Upgrade plan.

Block:

- New sales after an appropriate grace period.
- New products.
- New branches.
- New members.
- New workspaces.
- New applications.
- New imports.
- Advanced reports.

For Inventory, completely blocking sales can hurt a customer's business. Consider allowing a limited operational mode during grace period, then moving to read-only after the grace period.

15. Downgrade behavior

Before downgrade

Calculate conflicts:

Current:

- 5 workspaces
- 6 applications per workspace
- 15 members
- 5 branches

Target Standard:

- 3 workspaces
- 3 applications per workspace
- 10 members
- 3 branches

Show the user exactly what must change.

Rules

- Do not delete workspaces automatically.
- Do not delete products automatically.
- Do not delete members automatically.
- Archive excess resources if the user confirms.
- Allow the user to choose which resources remain active.
- Preserve data.
- Remove access to restricted features.
- Schedule downgrade at the end of the current period where possible.

16. Free-to-paid conversion strategy

Free plan should create value

The Free plan should allow users to experience:

- Product setup.
- Sales.
- Stock tracking.
- Basic reports.
- Customer credit.
- Basic receipts.

Upgrade triggers

The strongest upgrade triggers are likely:

- Need for a second workspace.
- Need for a second application.
- Need for more staff.
- Need for a second branch.
- Product limit reached.
- Transaction limit reached.
- CSV export.
- Advanced reports.
- Offline usage beyond free allocation.

Avoid aggressive interruption

Do not display upgrade prompts after every action. Use:

- Usage indicators.
- Contextual prompts.
- Email reminders.
- Upgrade banners.
- Limit-reached pages.
- Owner-only notifications.

17. Billing database architecture

Recommended relationship

```
users
  ↓
billingAccounts
  ↓
subscriptions
  ↓
plans
  ↓
entitlements
  ↓
workspaceProducts
```

Ownership question

If multiple users are workspace owners or billing managers, define a billing contact:

```
billingAccount.ownerUserId
billingAccount.workspaceId
billingAccount.billingEmail
```

For the initial version, a billing account can belong to one user and control several workspaces.

Future version:

```
billingAccounts
├─ multiple owners
├─ multiple billing managers
└─ many workspaces
```

18. Fastify pricing modules

```
src/modules/billing/
├─ billing.routes.ts
├─ billing.controller.ts
├─ billing.service.ts
├─ billing.repository.ts
├─ billing.schemas.ts
├─ entitlement.service.ts
├─ usage.service.ts
├─ plan.service.ts
├─ subscription.service.ts
├─ payment-provider.interface.ts
└─ paystack.provider.ts
```



```
|— billing-webhook.service.ts
|— billing.permissions.ts
```

Provider interface

```
interface PaymentProvider {
  initializeCheckout(input: InitializeCheckoutInput): Promise<CheckoutResult>
  verifyTransaction(reference: string): Promise<PaymentVerification>
  createSubscription(input: CreateSubscriptionInput): Promise<SubscriptionResult>
  cancelSubscription(providerSubscriptionId: string): Promise<void>
  getSubscription(providerSubscriptionId: string): Promise<ProviderSubscription>
}
```

This allows future support for Flutterwave or another provider without rewriting billing logic.

19. Required billing endpoints

```
GET    /v1/plans
GET    /v1/billing/account
GET    /v1/billing/subscription
GET    /v1/billing/usage
GET    /v1/billing/invoices
POST   /v1/billing/checkout
GET    /v1/billing/checkout/:reference
POST   /v1/billing/verify
POST   /v1/billing/change-plan
POST   /v1/billing/cancel
POST   /v1/billing/resume
PATCH /v1/billing/payment-method
POST   /v1/billing/webhooks/paystack
```

Plan-check endpoint

```
GET /v1/workspaces/:workspaceId/entitlements
```

Example response:

```
{
  "plan": {
    "key": "standard",
    "name": "Standard"
  },
  "entitlements": {
    "maxWorkspaces": 3,
    "maxAppsPerWorkspace": 3,
    "maxMembers": 10,
    "maxBranches": 3,
    "maxProducts": 5000,
    "maxMonthlyTransactions": 5000,
  }
}
```

```
    "apiAccess": false
  },
  "usage": {
    "workspaces": 1,
    "apps": 2,
    "members": 7,
    "branches": 2,
    "products": 2800,
    "monthlyTransactions": 3900
  }
}
```

20. Plan-specific application rules

Free

```
maxWorkspaces = 1
maxAppsPerWorkspace = 1
maxMembers = 2
maxBranches = 1
maxProducts = 500
maxMonthlyTransactions = 300
```

Standard

```
maxWorkspaces = 3
maxAppsPerWorkspace = 3
maxMembers = 10
maxBranches = 3
maxProducts = 5000
maxMonthlyTransactions = 5000
```

Premium

```
maxWorkspaces = 10
maxAppsPerWorkspace = unlimited
maxMembers = 50
maxBranches = 10
maxProducts = 25000
maxMonthlyTransactions = 25000
```

Avoid representing unlimited values as arbitrary large numbers. Use an explicit value:

```
limitType = "unlimited"
```

21. Pricing and workspace relationship

Free plan

A user may own only one workspace.

They may still be invited to another workspace by someone else, depending on your policy.

Important decision

Should the Free limit apply to:

1. Workspaces created by the user?
2. Workspaces owned by the user?
3. Workspaces the user belongs to?
4. Workspaces billed under the account?

Recommendation

Apply limits to workspaces owned or billed by the account, not every workspace the user joins.

Example:

Code X owns:

- Workspace A
- Workspace B

Code X is only a member of:

- Another company's Workspace C

The Free plan should not prevent Code X from joining Workspace C as an employee. The other company's plan should control Workspace C.

This is a crucial distinction.

Application limit

Similarly, distinguish:

- Applications activated in workspaces owned by the billing account.
- Applications the user merely accesses as a member.

A free user may join another company's Inventory workspace without using their own application entitlement.

22. Shared workspace billing

If a workspace has multiple members, the workspace's plan should determine access.

Example:

```
Workspace: Code X Stores  
Plan: Standard  
Members:  
- Owner  
- Manager  
- Sales Attendant  
- Accountant
```

The invited members do not each need separate subscriptions. They consume the workspace's member limit.

This is better suited to small businesses than charging every staff member separately.

23. Pricing user notifications

Usage notifications

- 50% limit reached.
- 80% limit reached.
- 90% limit reached.
- 100% limit reached.

Billing notifications

- Trial ending soon.
- Payment initiated.
- Payment successful.
- Payment failed.
- Payment retry.
- Grace period started.
- Subscription cancelled.
- Subscription renewed.
- Plan changed.
- Invoice available.

Workspace-owner notifications

Only owners and billing managers should receive billing notifications by default.

24. Audit events for pricing

Record:

```
billing.plan_selected
billing.checkout_started
billing.payment_initialized
billing.payment_verified
billing.payment_failed
billing.subscription_created
billing.subscription_renewed
billing.subscription_cancelled
billing.subscription_resumed
billing.plan_upgrade_requested
billing.plan_downgrade_requested
billing.plan_changed
billing.grace_period_started
billing.workspace_restricted
billing.entitlement_granted
billing.entitlement_revoked
billing.limit_reached
billing.usage_warning_sent
```

Never log:

- Card numbers.
- Authorization codes.
- Payment secrets.
- Raw provider credentials.

25. Pricing acceptance criteria

The pricing system is ready when:

- Free users can have only one owned workspace.
- Free users can activate only one application in their owned workspace.
- Standard users can have up to three workspaces.
- Standard users can activate up to three applications per workspace.
- Premium users can have up to ten workspaces.
- Premium users can activate all available applications.
- Product limits are enforced.
- Member limits are enforced.

- Branch limits are enforced.
- Transaction limits are enforced.
- Usage is displayed to workspace owners.
- Limits are checked server-side.
- Upgrade prompts appear when limits are reached.
- Payment is verified server-side.
- Webhooks are verified and deduplicated.
- Subscription state is updated correctly.
- Failed payments enter a grace period.
- Downgrades do not delete data.
- Cancellations preserve access until the effective end date.
- Entitlements are recalculated after plan changes.
- Workspace members do not need separate subscriptions.
- A free user can join another company's workspace without owning it.
- Billing actions generate audit logs.
- Billing owners receive notifications.
- Product applications do not contain duplicated plan logic.

26. Recommended implementation order

Phase 1: Plan configuration

- Create plan records.
- Create entitlement records.
- Define Free, Standard, and Premium limits.
- Build internal plan-management functions.
- Add plan comparison API.

Phase 2: Entitlement service

- Feature checks.
- Quantity-limit checks.
- Workspace checks.
- Product activation checks.
- Usage counters.
- Limit-reached errors.

Phase 3: Billing screens

- Pricing page.
- Billing page.
- Usage page.
- Upgrade modal.
- Checkout page.
- Payment-success page.
- Payment-failure page.

Phase 4: Paystack integration

- Plan creation.
- Checkout initialization.
- Server-side verification.
- Webhook handling.
- Subscription updates.
- Renewal handling.
- Payment failure handling.

Phase 5: Workspace integration

- Workspace creation limits.
- Product activation limits.
- Member limits.
- Branch limits.
- Product limits.
- Transaction limits.

Phase 6: Subscription lifecycle

- Trial.
- Active subscription.
- Past due.
- Grace period.
- Cancellation.
- Downgrade.
- Suspension.
- Restoration.

Phase 7: Reporting and support

- Usage dashboard.
- Billing history.
- Invoice records.
- Billing audit logs.
- Admin support tools.
- Manual entitlement correction with audit trail.

Final recommendation

Use this commercial model:

Free:

1 workspace
1 application
1 branch
2 members
500 products
300 monthly transactions

Standard:

3 workspaces
3 applications per workspace
3 branches per workspace
10 members per workspace
5,000 products
5,000 monthly transactions
¥7,500/month

Premium:

10 workspaces
All applications
10 branches per workspace
50 members per workspace
25,000 products
25,000 monthly transactions
¥20,000/month

The most important pricing design decision is to bill **workspaces**, not individual users. A user can belong to many other people's workspaces without consuming their personal workspace limit. The plan should control workspaces and products owned by the billing account, while workspace members receive access according to their assigned role.

This gives Orvionhub a fair and scalable model:

One account

↓

Owns or bills multiple workspaces

↓
Each workspace activates products
↓
Each product has members and permissions
↓
Plan entitlements control limits

Use centralized entitlements rather than embedding pricing checks inside Inventory or Task Management. That way, future applications can reuse the same pricing, billing, usage, upgrade, downgrade, and subscription infrastructure.

✱

1. <https://www.dechsoft.com/pricing>
2. <https://inkeepx.com/blog/how-much-does-pos-software-cost-in-nigeria-2026-guide>
3. <https://shopkeeperpos.com/pricing>
4. <https://swiftpos.ng/blog/inventory-management-software-for-supermarkets-in-nigeria-the-complete-2026-guide/>
5. <https://paystack.com/docs/payments/recurring-charges/>
6. <https://paystack.com/docs/payments/subscriptions/>
7. <https://docs.stripe.com/billing/subscription-pricing>
8. <https://docs.stripe.com/billing/subscriptions/overview>
9. <https://paystack.com/docs/payments/>
10. <https://github.com/PaystackHQ/documentation/blob/master/recurring-payments/subscriptions.md>
11. <https://support.paystack.com/en/articles/2133058>
12. <https://support.paystack.com/en/articles/2132546>
13. <https://docs.stripe.com/billing/entitlements>
14. <https://docs.stripe.com/billing/entitlements?dashboard-or-api=api>
15. <https://stripe.com/legal/subscription-policy>